Assessment Task 3: Data analytics in action

JIWEI JIANG 25633037

Data Mining and Processing 2
Model Explore 8
KNIME workflow 8
Decision Tree 8
KNN 10
MLP 12
SVM 14
Random Forest 17
Tree Ensemble 18
Debugging model 22
The best classifier 23
Reflection 24
References 26

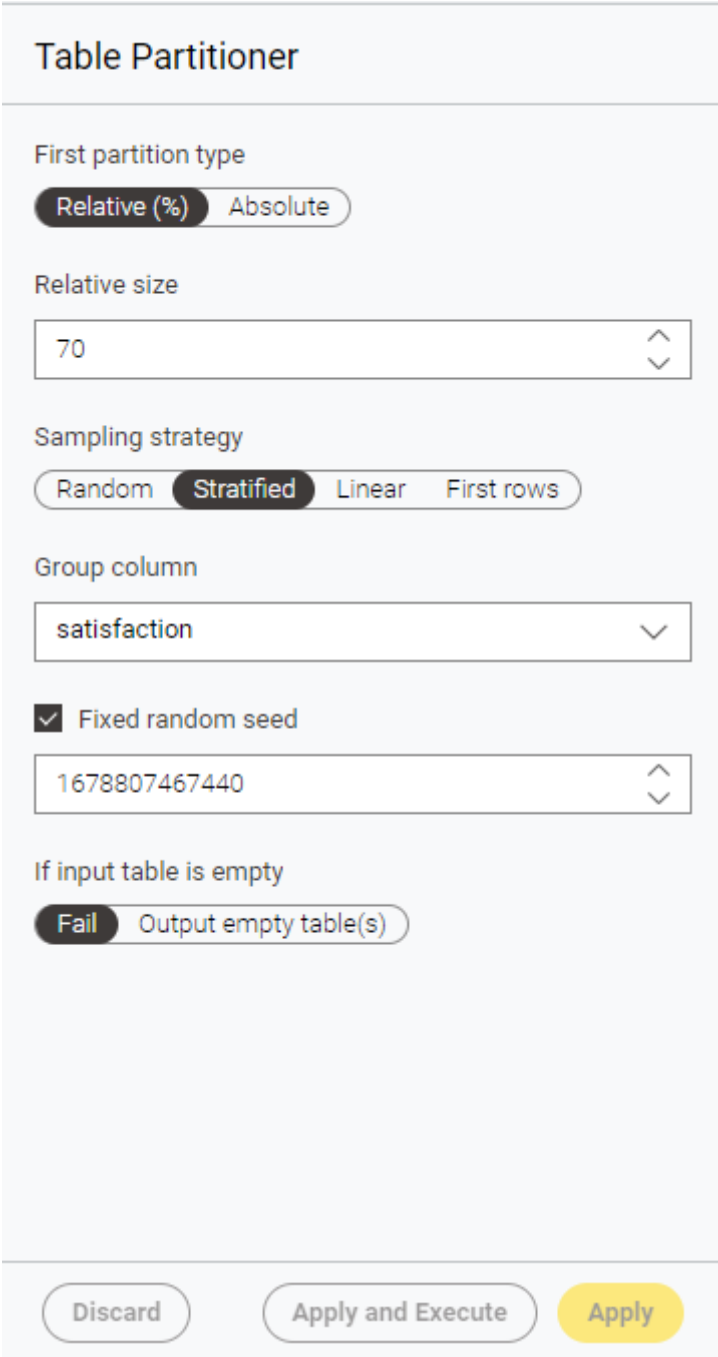
Data Mining and Processing

The goal of this project is to predict passenger satisfaction (satisfied or neutral/dissatisfied) by analyzing service experience data of airline passengers. The dataset contains over 80000 records, covering features such as age, flight distance, in-flight entertainment, dining, and seat comfort.

In the KNIME platform, various classification algorithms are applied, including decision trees, KNN, SVM, MLP, and random forests, and the model quality is improved through preprocessing steps such as data cleaning, standardization, and category encoding. The model performance is evaluated through cross validation with metrics such as accuracy, recall, F1 score, and AUC.

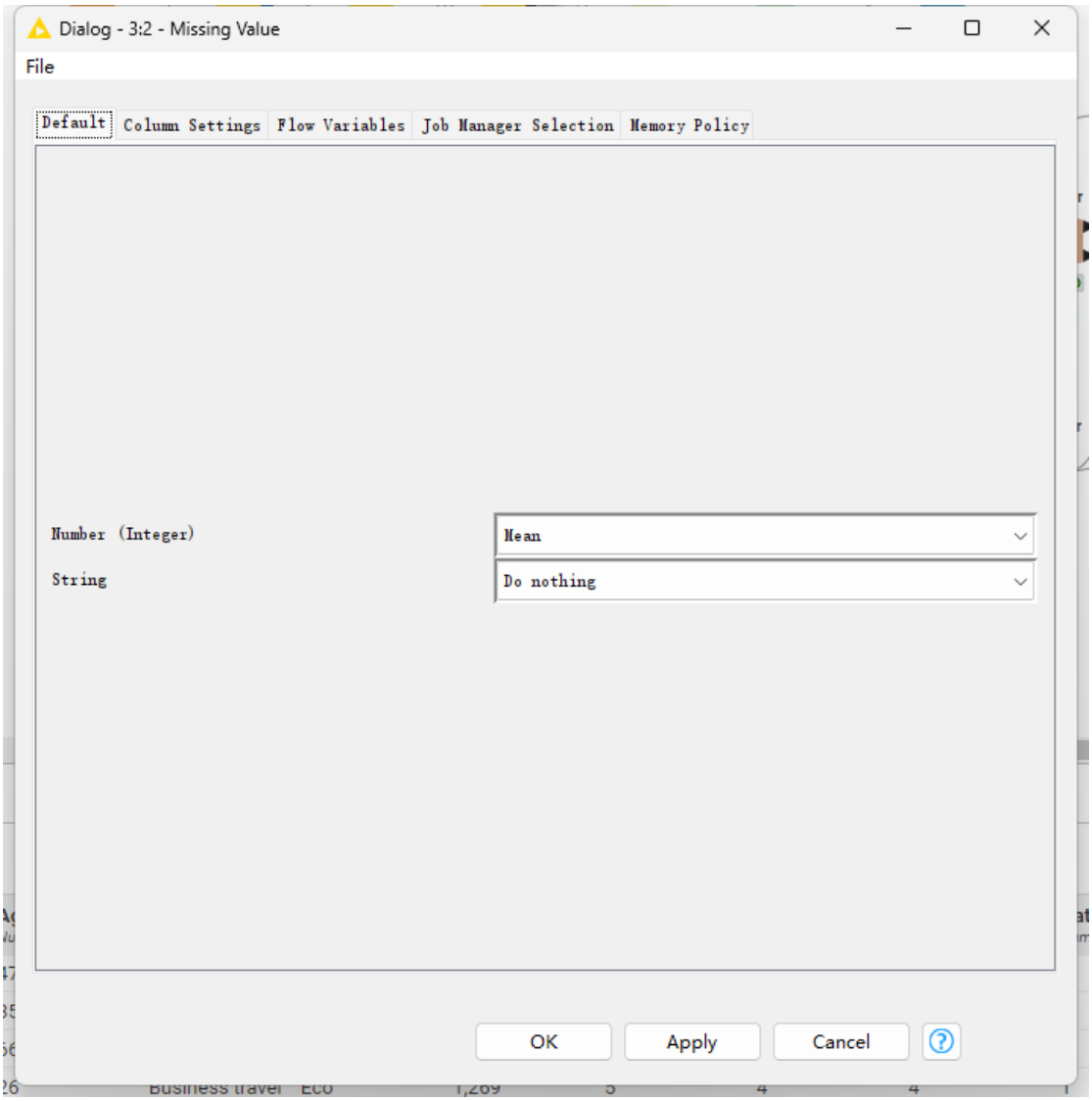
The purpose of this data mining task is to identify the key factors that affect passenger satisfaction and determine the most effective predictive model to provide data support for airline service improvement.

Table Partitioner:



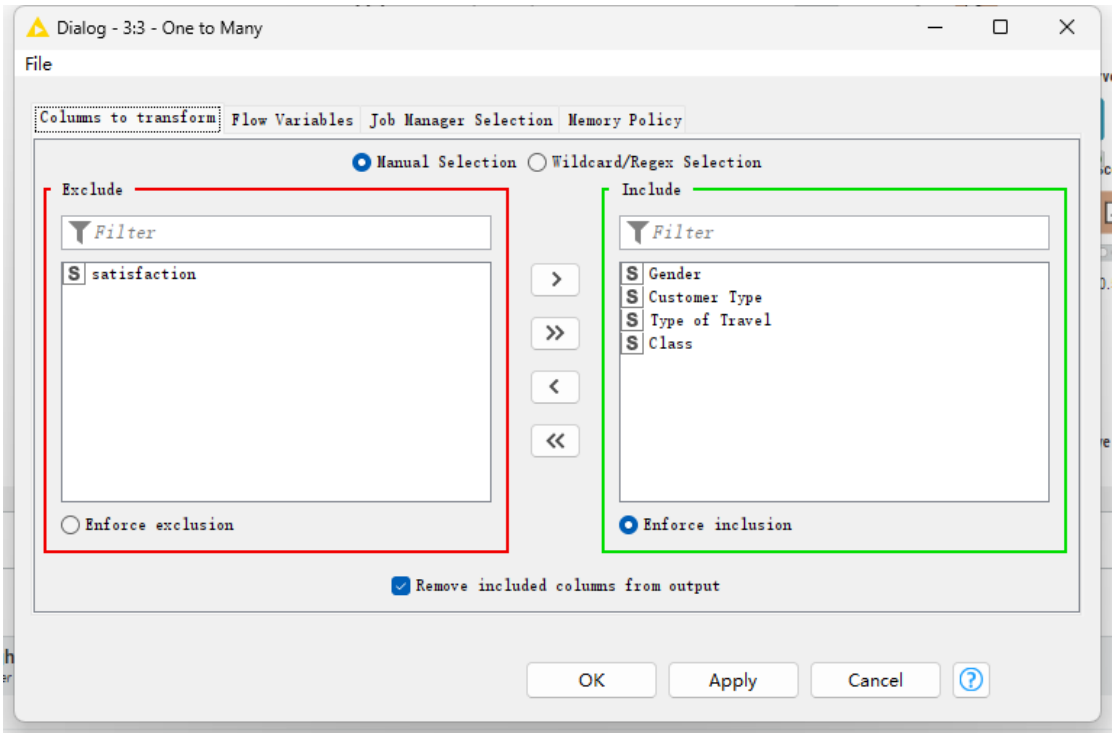
I use 30% table to train and 70% to test.

Missing Value:



Take mean to missing number and do nothing to missing string. Some missing number will cause error when running other node.

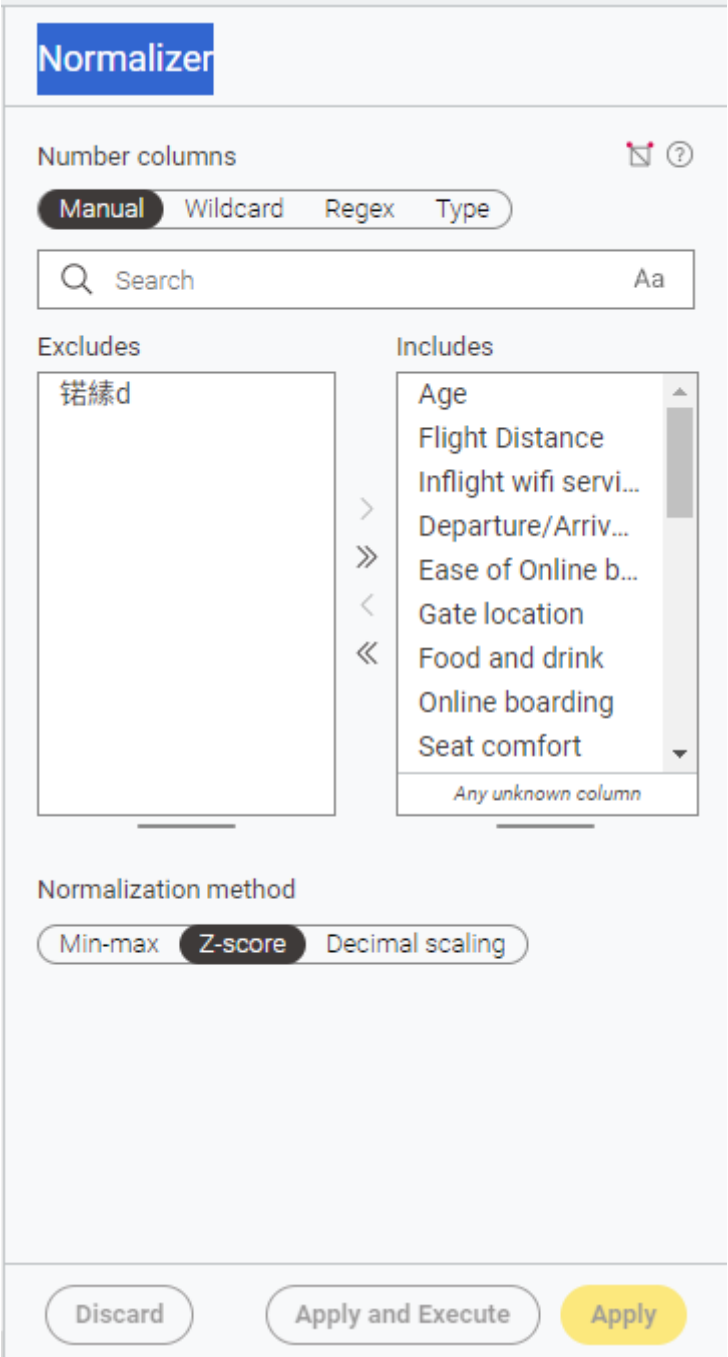
One to Many



Many machine learning algorithms, such as KNN, SVM, neural networks, etc., are unable to directly handle categorical variables of text types.

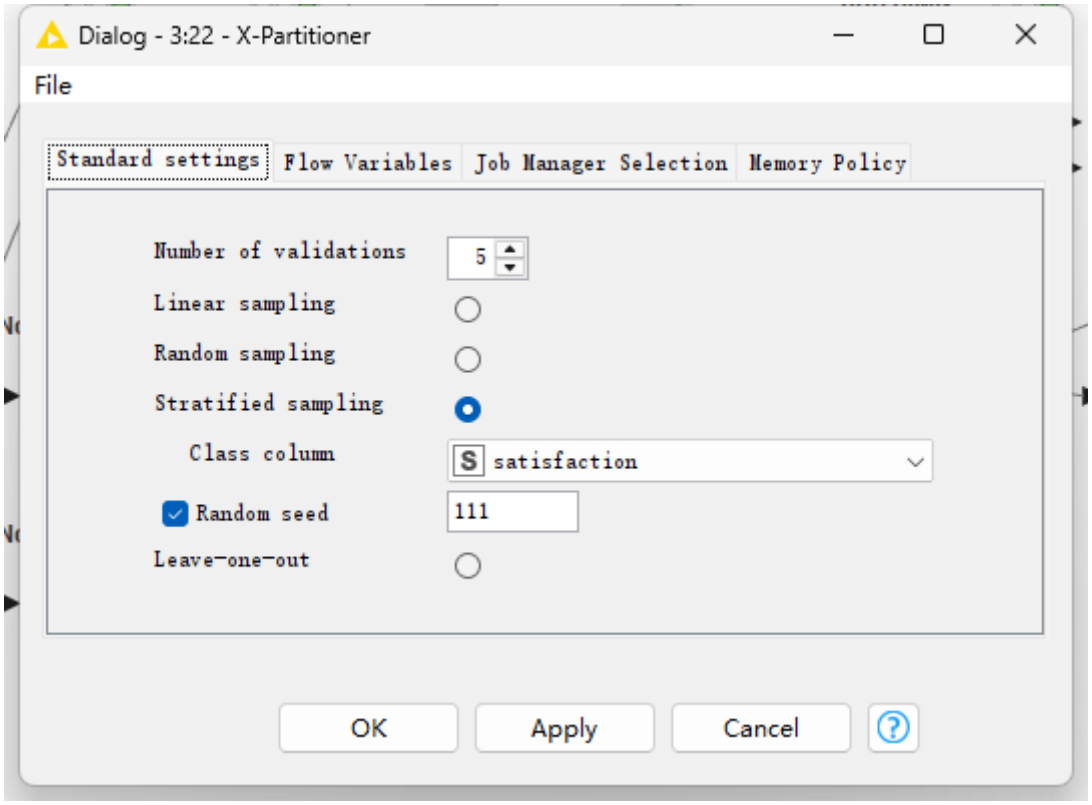
Therefore, we need to use the One to Many node to convert them into numerical columns.

Normalizer



There are different levels of data in the table, and if directly input into the model, it will lead to distance based algorithms (such as KNN, SVM) being dominated by large numerical features, slow neural network training or difficult convergence, and uneven distribution of weights or information gains. Therefore, I used the z-score method for normalization

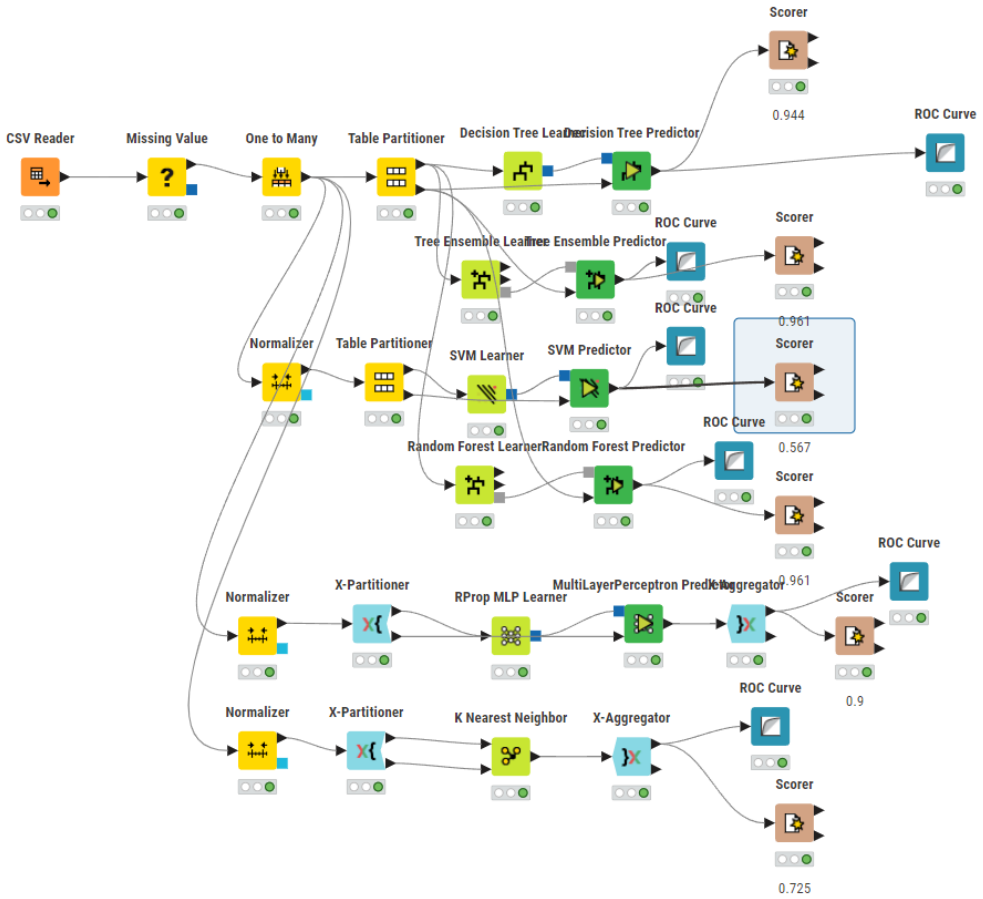
Cross-Validation



Cross Validation is used to objectively evaluate the generalization ability of a model, that is, to determine whether the model's performance on unknown data is stable and reliable. I am using both the MLP model and KNN model for cross validation evaluation. The data is divided into 5 parts, with 1 part used for testing and the other 4 parts used for training, for a total of 5 training sessions. Each model has undergone multiple rounds of training and validation, and the final Scorer and ROC Curve nodes demonstrate the average performance metrics.

Model Explore

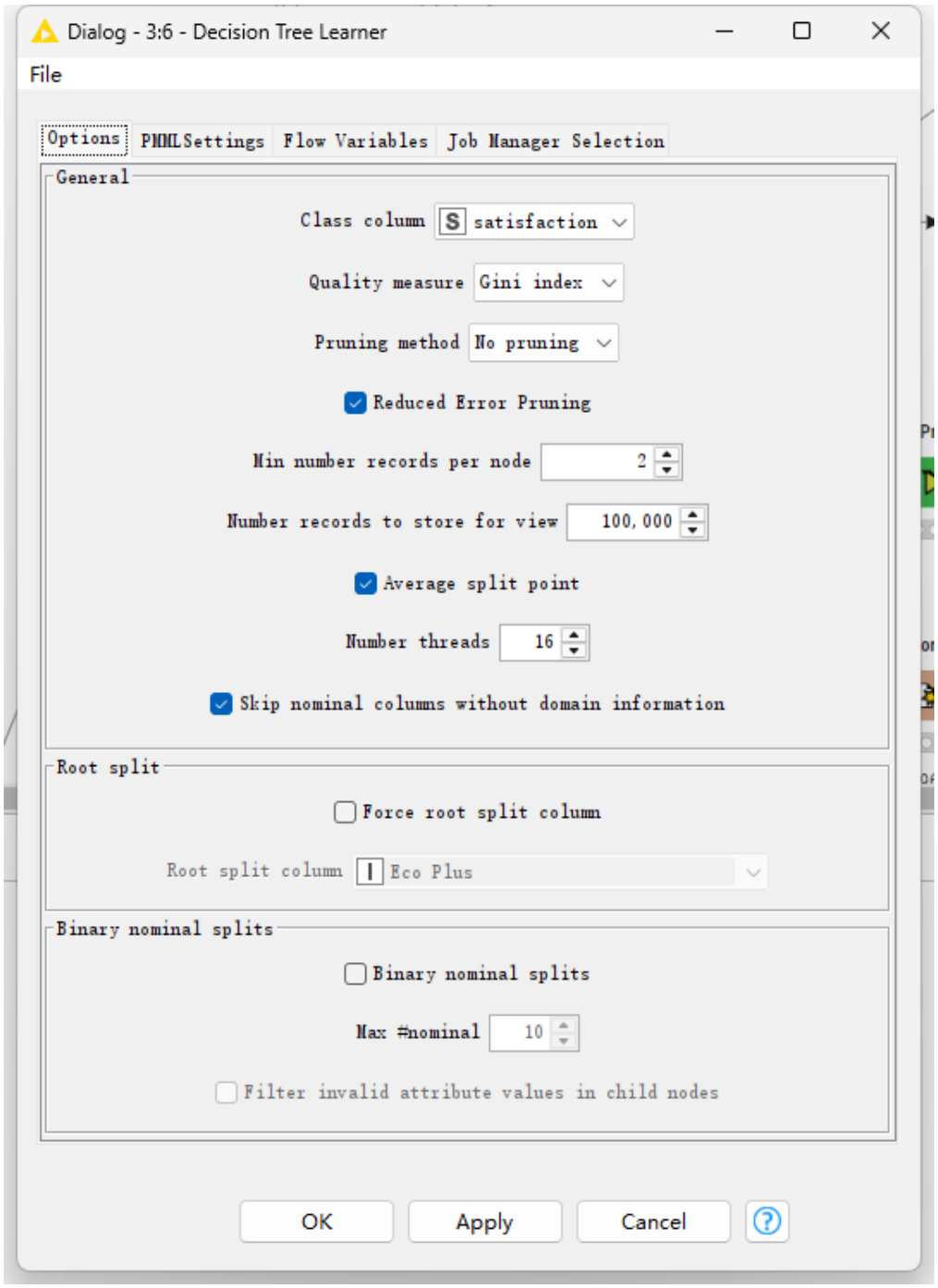
KNIME workflow



Decision Tree

Decision Tree is a commonly used supervised learning model that can be used for classification and regression problems. It continuously divides the dataset into smaller subsets, ultimately forming a tree like structure for making predictions (IBM, 2021c).

Setting of Model



Quality measure: Gini index, suitable for classification tasks.

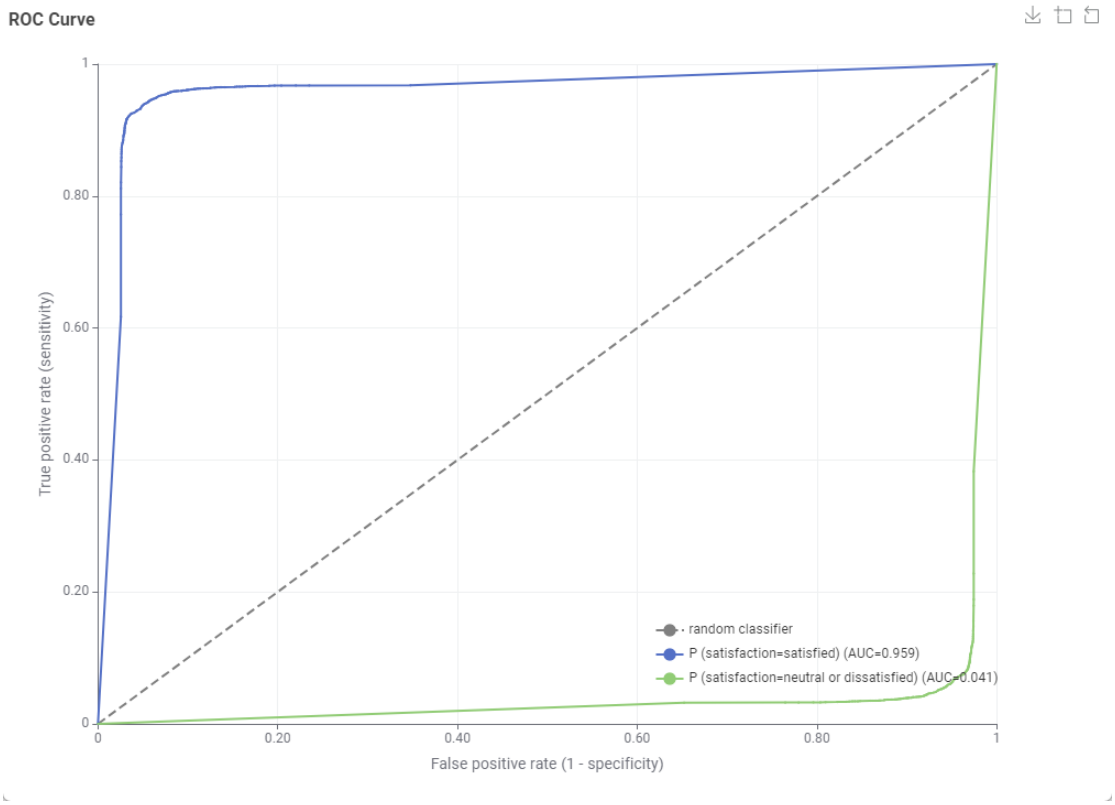
Reduced Error Pruning: A validation set pruning method to reduce overfitting. The model will remove useless branches after generation to improve generalization ability.

Min number records per node: The minimum number of samples allowed for each node, with smaller trees being deeper and more complex; A larger value makes the tree simpler but may

result in underfitting.

Average split point: Take the average of the splitting points of continuous features to avoid unreasonable partitioning caused by an outlier.

After multiple debugging of the model, the highest accuracy result obtained is:



Rows: 3 | Columns: 11

Table Statistics

#	RowID	TruePositives Number (Integer)	FalsePositives Number (Integer)	TrueNegatives Number (Integer)	FalseNegatives Number (Integer)	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's kappa Number (Float)
1	satisfie	10078	673	13458	728	0.933	0.937	0.933	0.952	0.935	?	?
2	neutral	13458	728	10078	673	0.952	0.949	0.952	0.933	0.951	?	?
3	Overall	?	?	?	?	?	?	?	?	?	0.944	0.886

The AUC value is 0.959, indicating that the model has strong discriminative ability; The overall accuracy is 94.4%, with a balanced recall and precision.

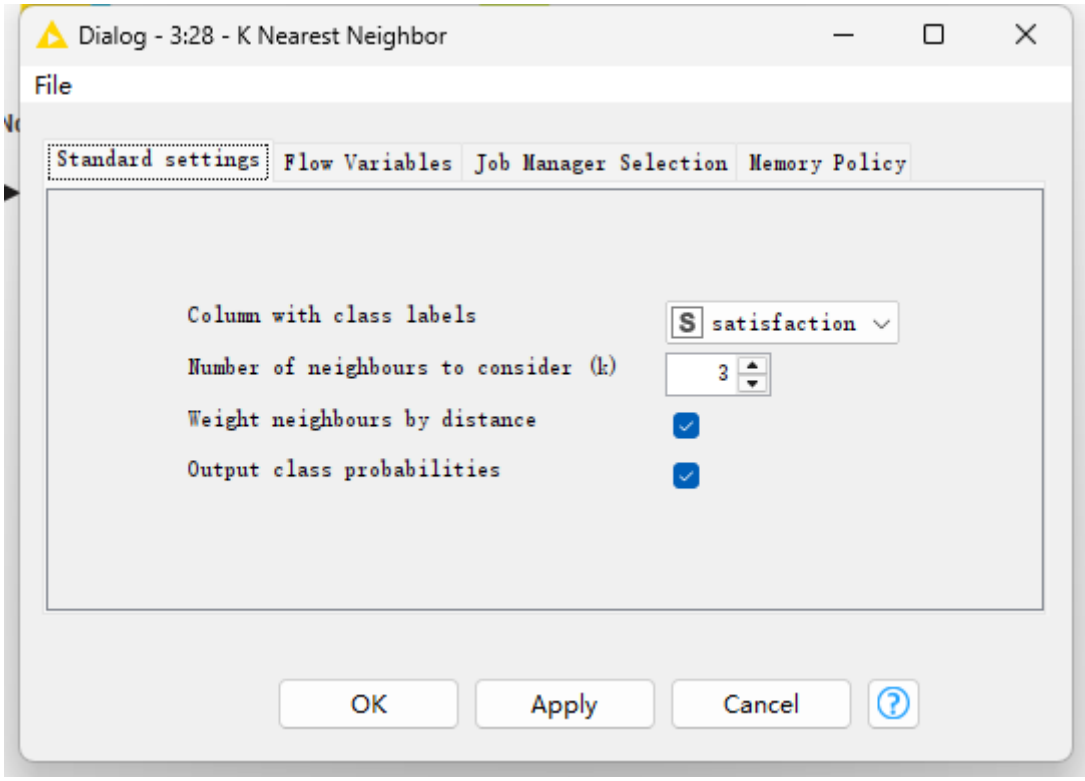
KNN

KNN (K-Nearest Neighbors) is a distance based supervised learning algorithm that can be used for classification and regression. It does not rely on complex model formulas, but makes predictions by comparing the distances between samples (IBM, 2021a).

It is simple and intuitive, does not require a training stage, is effective for non-linear data

distributions, and can be used for multi classification and regression tasks. It has a large computational load in the prediction stage and is sensitive to feature scales.

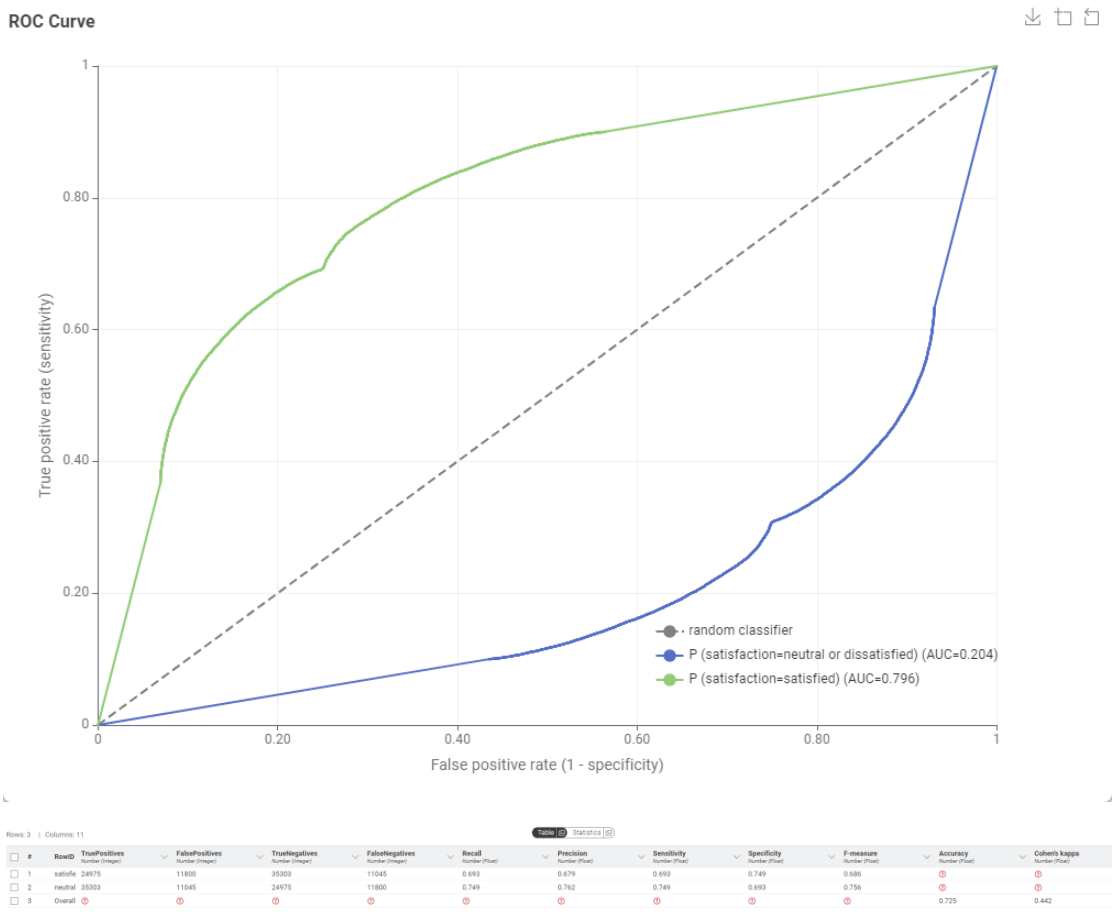
Setting of Model



Number of neighbours to consider (k): The number of nearest neighbor samples considered by the algorithm during prediction.

Weight neighbours by distance: Neighbors who are closer have higher weights and greater impact.

After multiple debugging of the model, the highest accuracy result obtained is:



#	RowID	TruePositives Number (Integer)	FalsePositives Number (Integer)	TrueNegatives Number (Integer)	FalseNegatives Number (Integer)	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's kappa Number (Float)
1	satisfie	24975	11800	35303	11045	0.693	0.679	0.693	0.749	0.686	?	?
2	neutral	35303	11045	24975	11800	0.749	0.762	0.749	0.693	0.756	?	?
3	Overall	?	?	?	?	?	?	?	?	?	0.725	0.442

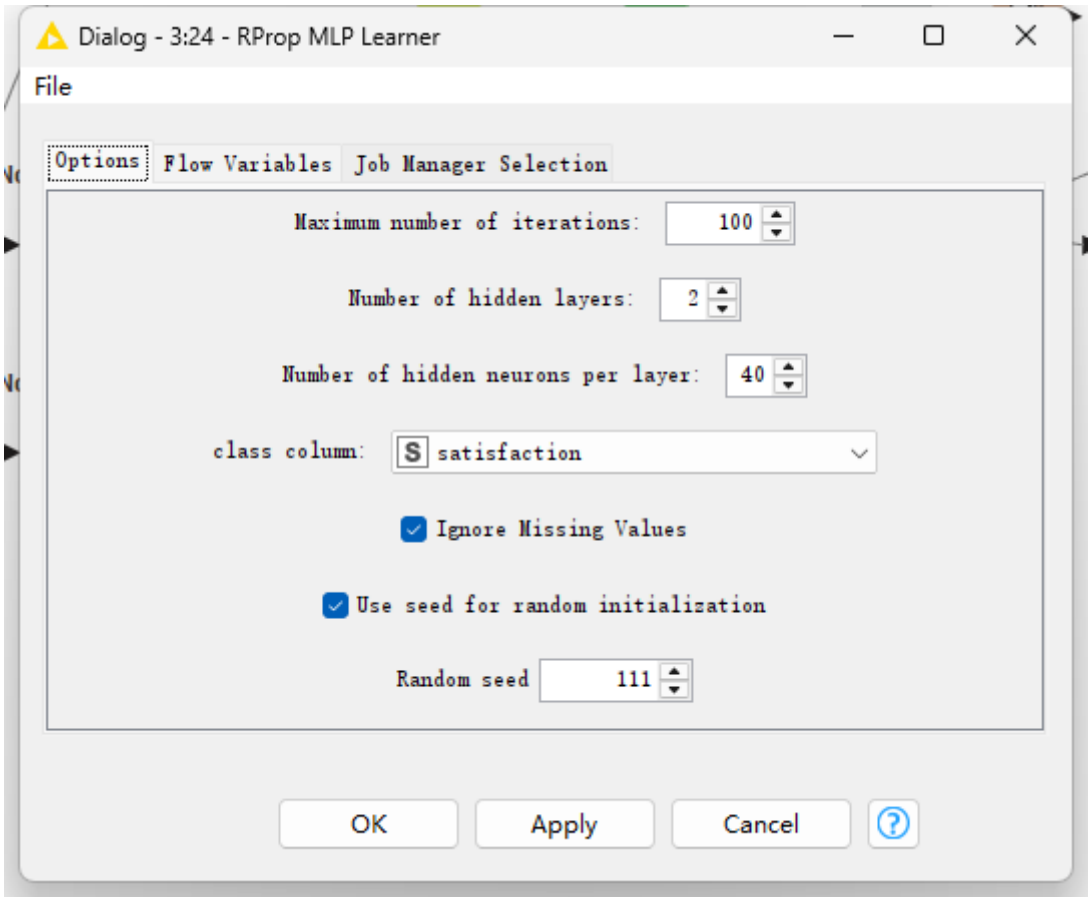
The overall accuracy of the model is about 72.5%, and its recognition of the "neutral or dissatisfied" category (Recall=0.749) is slightly better than that of "satisfied" (Recall=0.693). The model can distinguish most samples, but there is still some confusion.

MLP

MLP (Multi Layer Perceptron) is a type of feedforward neural network that belongs to supervised learning algorithms. It is composed of multiple layers of neurons and learns the nonlinear relationship between input features and output labels through weights and activation functions (Jaiswal, 2024).

It performs well on large data, has strong generalization ability, and can be used for classification, regression, and probability prediction, but it is prone to overfitting and has a long training time.

Setting of Model

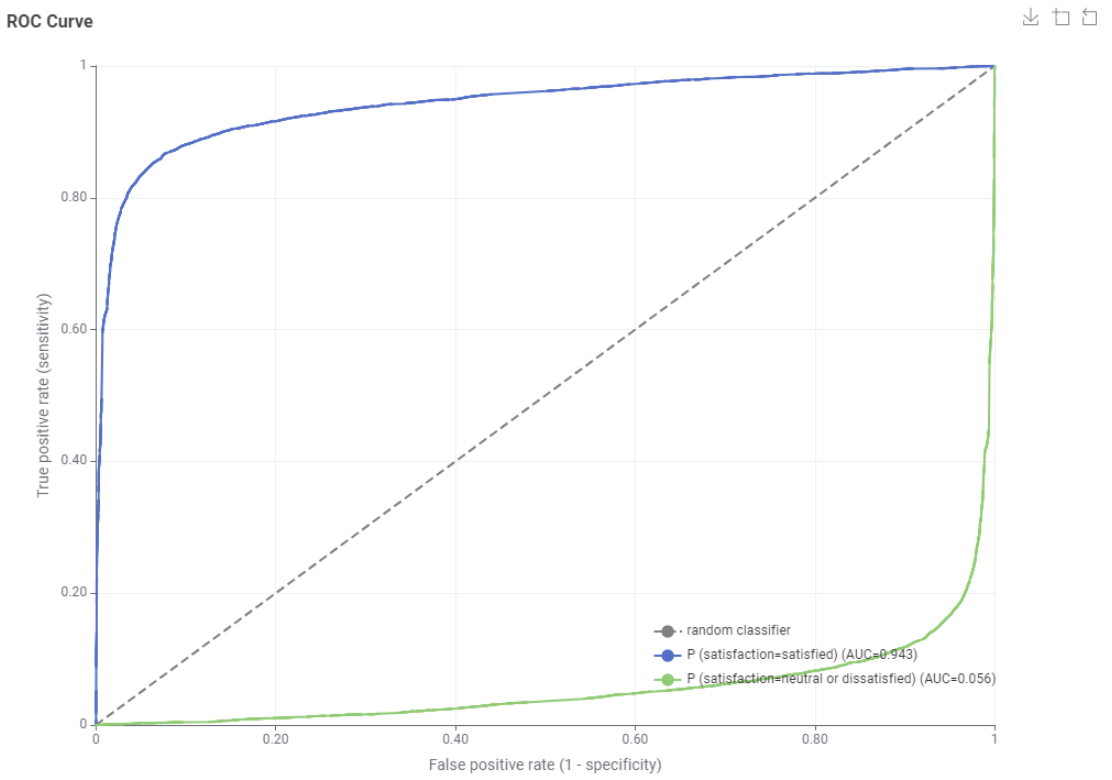


Maximum number of iterations: The maximum number of iterations (epochs) for model training, where each iteration will traverse all training samples and update weights.

Number of hidden layers: Number of hidden layers. Each layer can capture more complex nonlinear relationships.

Number of hidden neurons per layer: The number of neurons in each layer. Control the learning capacity of the model.

After multiple debugging of the model, the highest accuracy result obtained is:



#	RowID	TruePositives Number (Integer)	FalsePositives Number (Integer)	TrueNegatives Number (Integer)	FalseNegatives Number (Integer)	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's kappa Number (Float)
1	satisfie	30400	2707	44396	5620	0.844	0.918	0.844	0.943	0.88	?	?
2	neutral	44396	5620	30400	2707	0.943	0.888	0.943	0.844	0.914	?	?
3	Overall	?	?	?	?	?	?	?	?	?	0.9	0.794

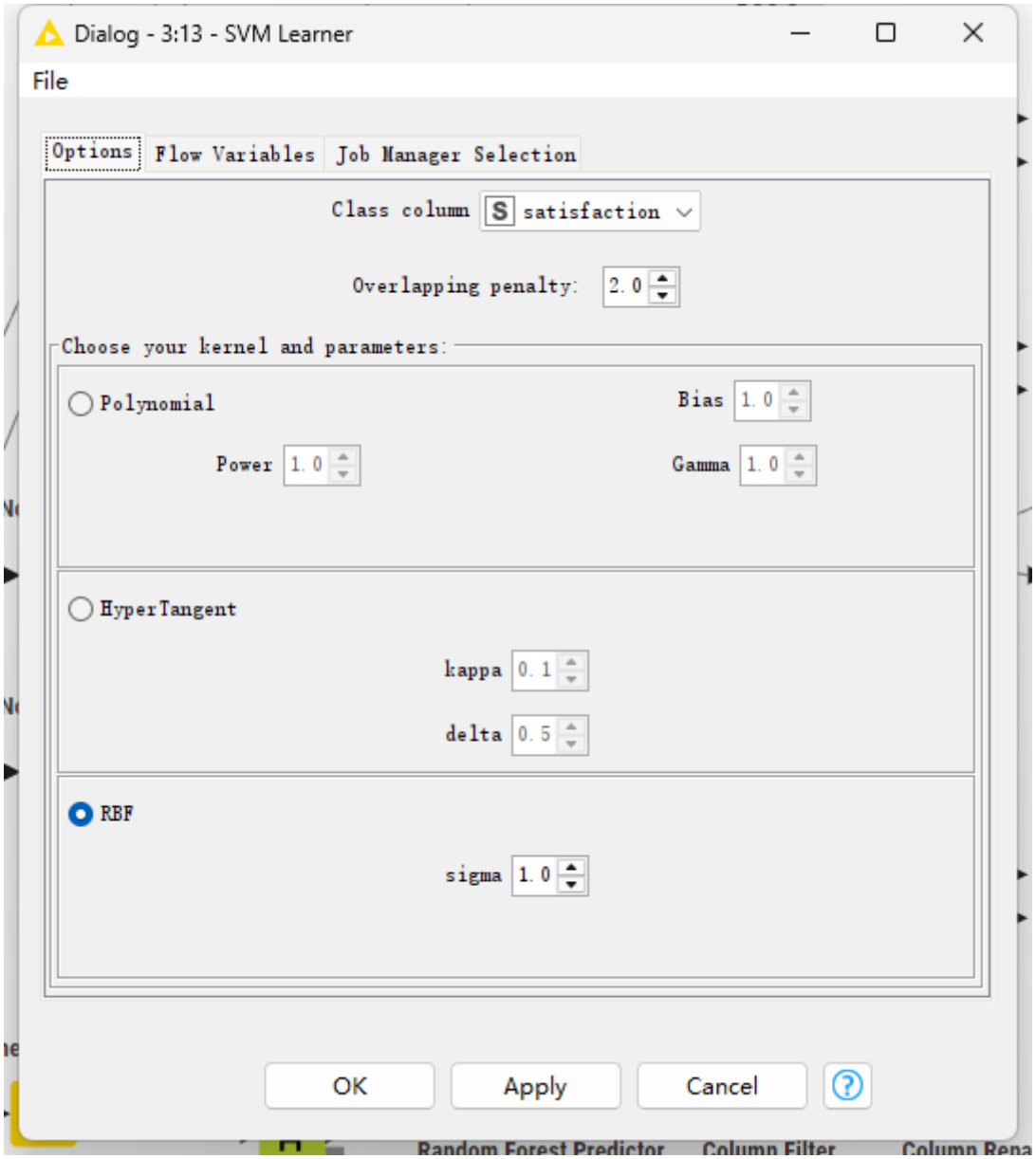
The blue curve is almost close to the upper left corner, indicating that the model can efficiently identify the "satisfied" category. The green curve is almost along the coordinate axis, indicating that the judgment of "dissatisfied" passengers is relatively insensitive. Both types of F-measures are above 0.88, indicating that the model performs well in balancing accuracy and recall, with an overall accuracy of 90%.

SVM

SVM (Support Vector Machine) is a classic supervised machine learning algorithm mainly used for classification and regression. Its core idea is to find an "optimal separation line" (or hyperplane) in the feature space, allowing samples of different categories to be separated as much as possible, and maximizing the distance between the samples closest to this line (i.e., the "support vector") (IBM, 2023).

It is suitable for high-dimensional data, can handle nonlinear relationships, has strong generalization ability, can be flexibly adjusted using different kernel functions, and performs well on small sample data.

Setting of Model:

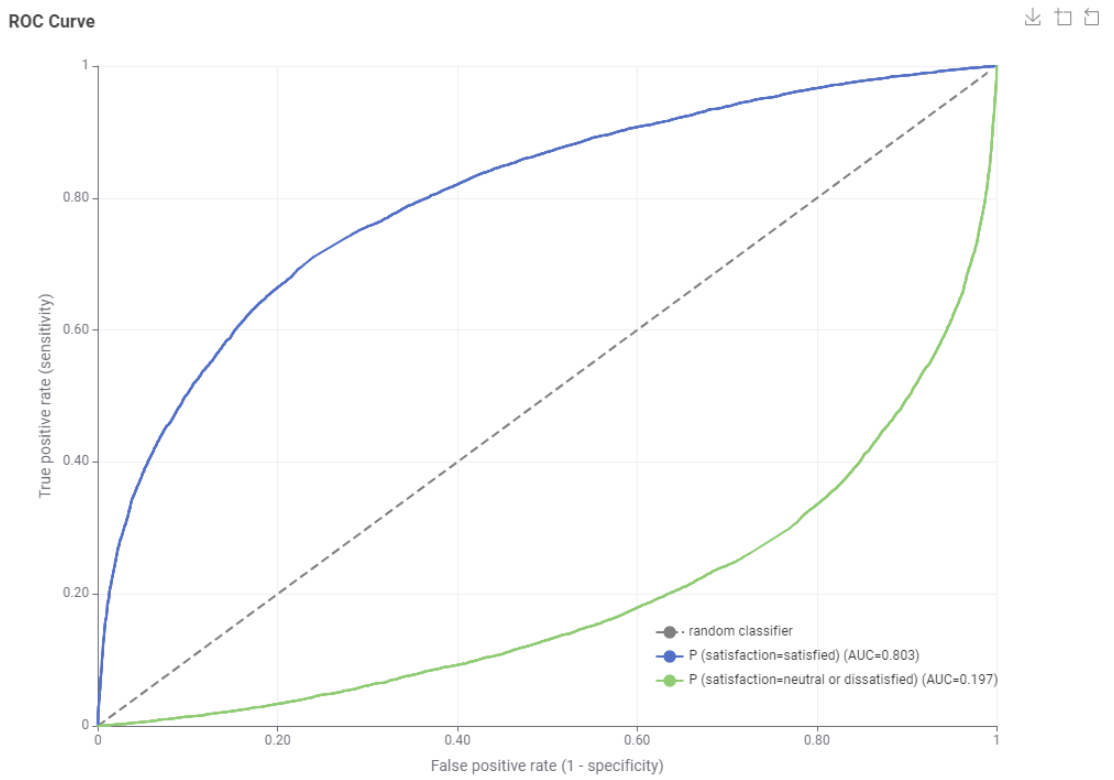


Overlapping penalty (C): Punishment coefficient, also known as regularization parameter C. It controls the model's tolerance for misclassified samples, with larger values emphasizing classification accuracy (prone to overfitting) and smaller values emphasizing larger intervals (stronger generalization)

Kernel and parameters: Polynomial maps data to a polynomial space, suitable for non-linear but clearly regular data, and can be used when the classification boundary is non-linear but can be approximated by a curve; HyperTangent is suitable for symmetric bidirectional feature relationships and is generally used in experimental models; RBF maps samples to high-

dimensional space, which can effectively handle complex nonlinear data and is suitable for complex datasets.

After multiple debugging of the model, the highest accuracy result obtained is:



Rows: 3 | Columns: 11

Table | Statistics

#	RowID	TruePositives Number (Integer)	FalsePositives Number (Integer)	TrueNegatives Number (Integer)	FalseNegatives Number (Integer)	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's kappa Number (Float)
1	satisfic	13	0	14131	10793	0.001	1	0.001	1	0.002	?	?
2	neutral	14131	10793	13	0	1	0.567	1	0.001	0.724	?	?
3	Overall	?	?	?	?	?	?	?	?	?	0.567	0.001

The model almost predicts all samples as the "neutral" category (Recall=1.0), while the Recall of the "satisfied" category is only 0.001. Although the Precision of the "satisfied" class is 1.0 (meaning that what is predicted as satisfactory is indeed satisfactory), due to the extremely low recognition rate, the F1 score is only 0.002.

Through the evaluation of the Random Forest model, I found that the overall accuracy of the model is average, but its performance is limited in imbalanced data categories. Although the ROC curve shows that the model has a certain recognition ability for the "satisfied" category (AUC=0.803), its recall rate is extremely low, indicating that the model has not fully learned minority class features. The generalization performance and fairness of the model can be

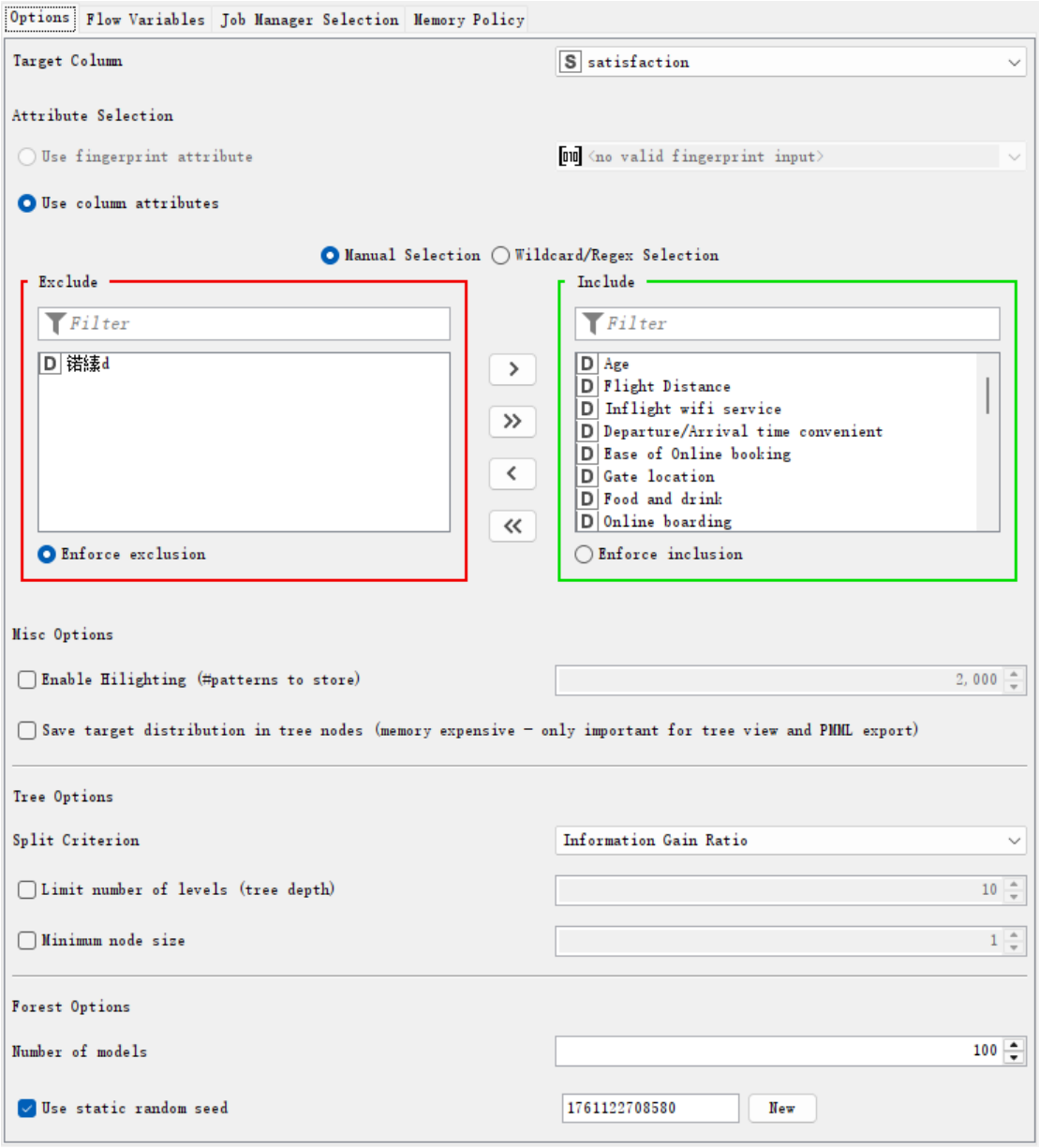
improved through data balancing and parameter optimization in the future.

Random Forest

Random Forest is an ensemble learning based algorithm that uses "multiple decision trees voting together" for classification or regression (IBM, 2021b).

Random Forest has strong anti-overfitting ability, can handle high-dimensional data, can measure which features can most affect the results, multiple tree voting, strong model robustness, and is usually more stable than a single decision tree

Setting of Model:



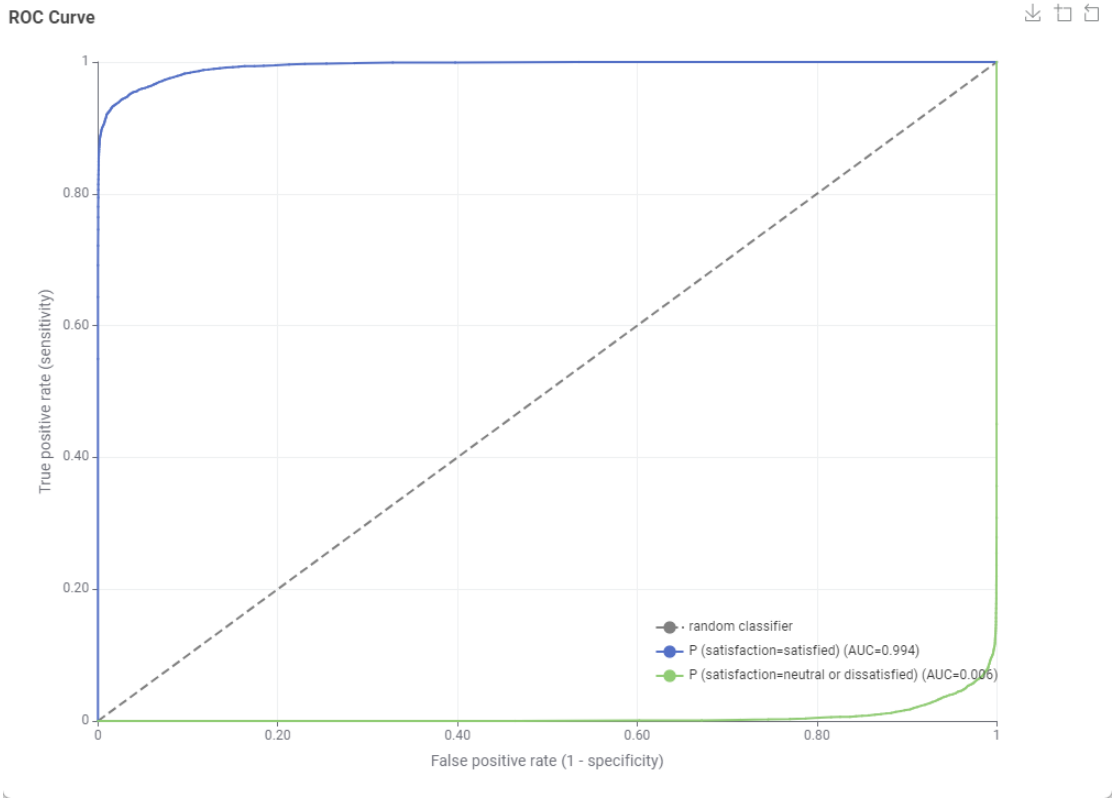
Split Criterion: The criteria used by decision trees during node splitting.

Limit number of levels (tree depth): Limit the maximum depth of each tree (to prevent overfitting).

Minimum node size: Minimum sample size. If the node sample is less than this value, it will no longer split.

Number of models: The number of trees contained in the forest

After multiple debugging of the model, the highest accuracy result obtained is:



Rows: 3 | Columns: 11

Table | Statistics

#	RowID	TruePositives Number (Integer)	FalsePositives Number (Integer)	TrueNegatives Number (Integer)	FalseNegatives Number (Integer)	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's kappa Number (Float)
1	satisfie	10118	290	13841	688	0.936	0.972	0.936	0.979	0.954	?	?
2	neutral	13841	688	10118	290	0.979	0.953	0.979	0.936	0.966	?	?
3	Overall	?	?	?	?	?	?	?	?	?	0.961	0.92

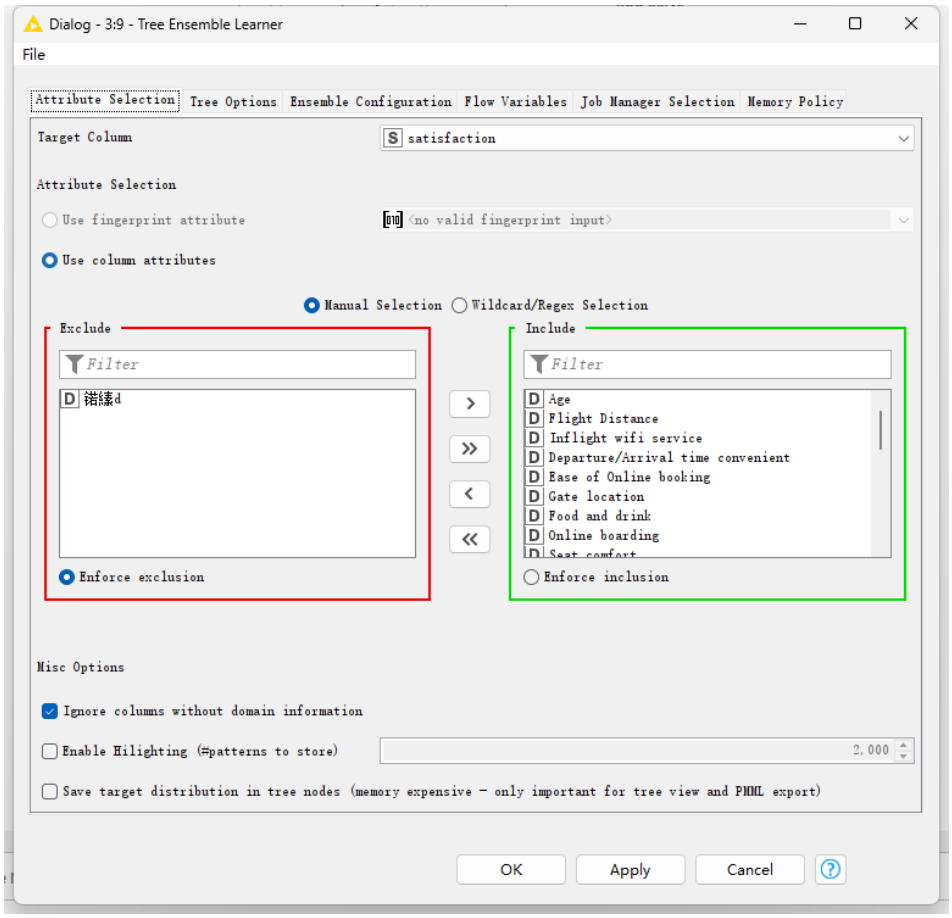
The overall classification accuracy of the model is as high as 96.1%, and the model can stably distinguish between satisfied and dissatisfied passengers. Kappa 0.92 measures the difference between model prediction and random prediction, indicating that the model has extremely high reliability.

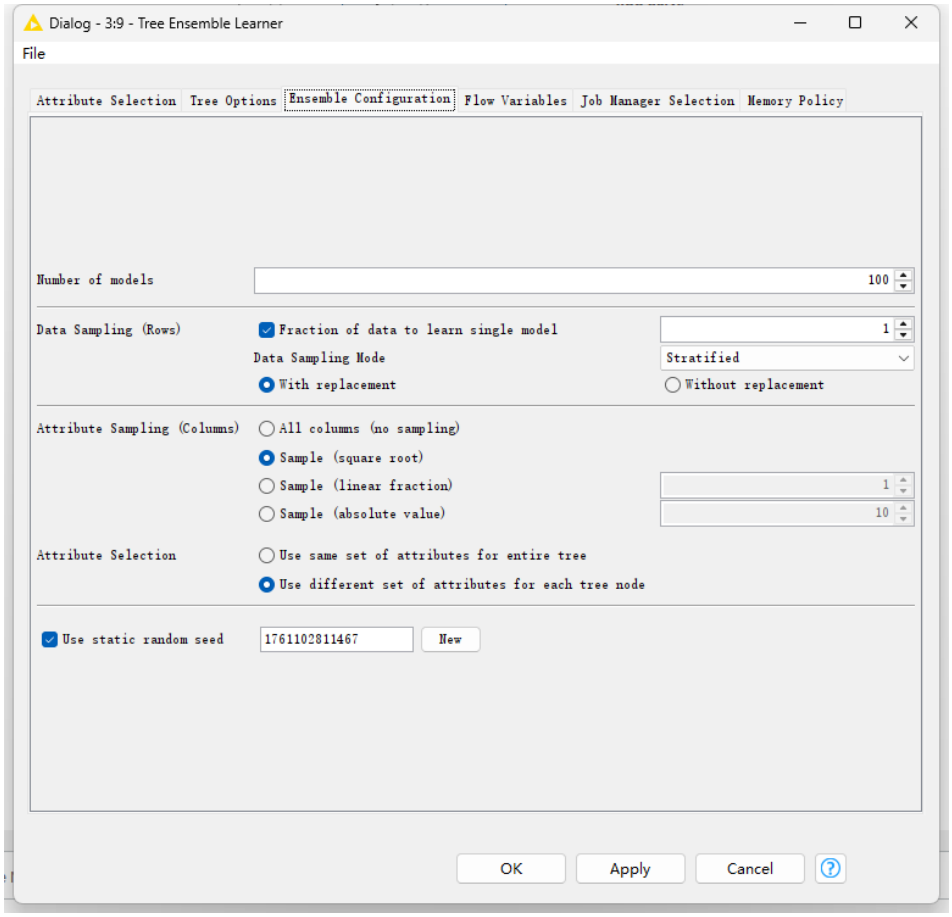
Tree Ensemble

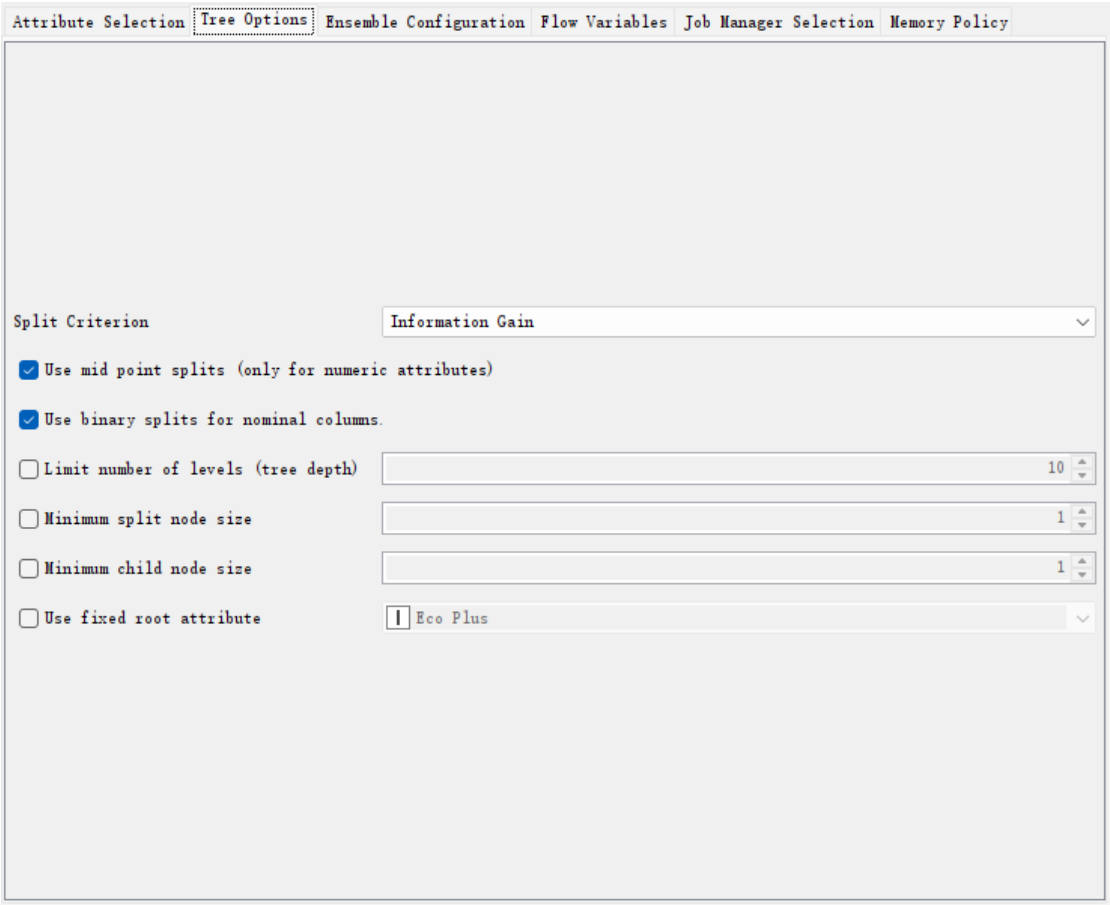
Tree Ensemble is a type of ensemble learning model based on the combination of multiple

decision trees, and Random Forest and Gradient Boosted Trees (GBM/XGBoost/LightGBM) belong to its specific types (scikit-learn, 2012).

Setting of Model:




Dialog - 3:9 - Tree Ensemble Learner
File
Attribute Selection
Tree Options
Ensemble Configuration
Flow Variables
Job Manager Selection
Memory Policy
Number of models
100
Data Sampling (Rows)
Fraction of data to learn single model
1
Data Sampling Mode
Stratified
With replacement
Without replacement
Attribute Sampling (Columns)
All columns (no sampling)
Sample (square root)
Sample (linear fraction)
1
Sample (absolute value)
10
Attribute Selection
Use same set of attributes for entire tree
Use different set of attributes for each tree node
Use static random seed
1761102811467
New
OK
Apply
Cancel

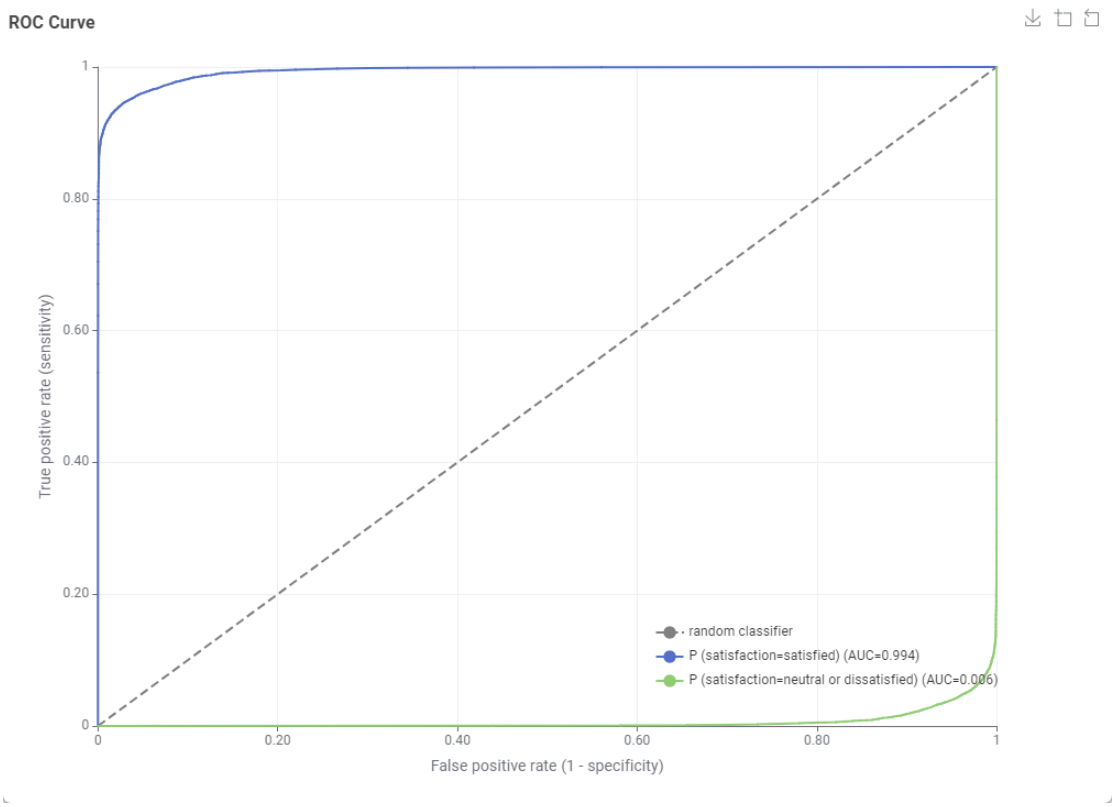

Attribute Selection
Tree Options
Ensemble Configuration
Flow Variables
Job Manager Selection
Memory Policy
Split Criterion
Information Gain
Use mid point splits (only for numeric attributes)
Use binary splits for nominal columns.
Limit number of levels (tree depth)
10
Minimum split node size
1
Minimum child node size
1
Use fixed root attribute
Eco Plus

Split Criterion: Measuring the information improvement after feature partitioning, commonly used for classification tasks.

Use midpoint splits: Numerical features are divided by midpoints to improve the accuracy of continuous variable partitioning.

Use binary splits for nominal columns: Make the model only consider two classification paths at a time (more robust).

After multiple debugging of the model, the highest accuracy result obtained is:



▶ 1: Confusion matrix ▶ 2: Accuracy statistics Flow Variables

Rows: 3 | Columns: 11

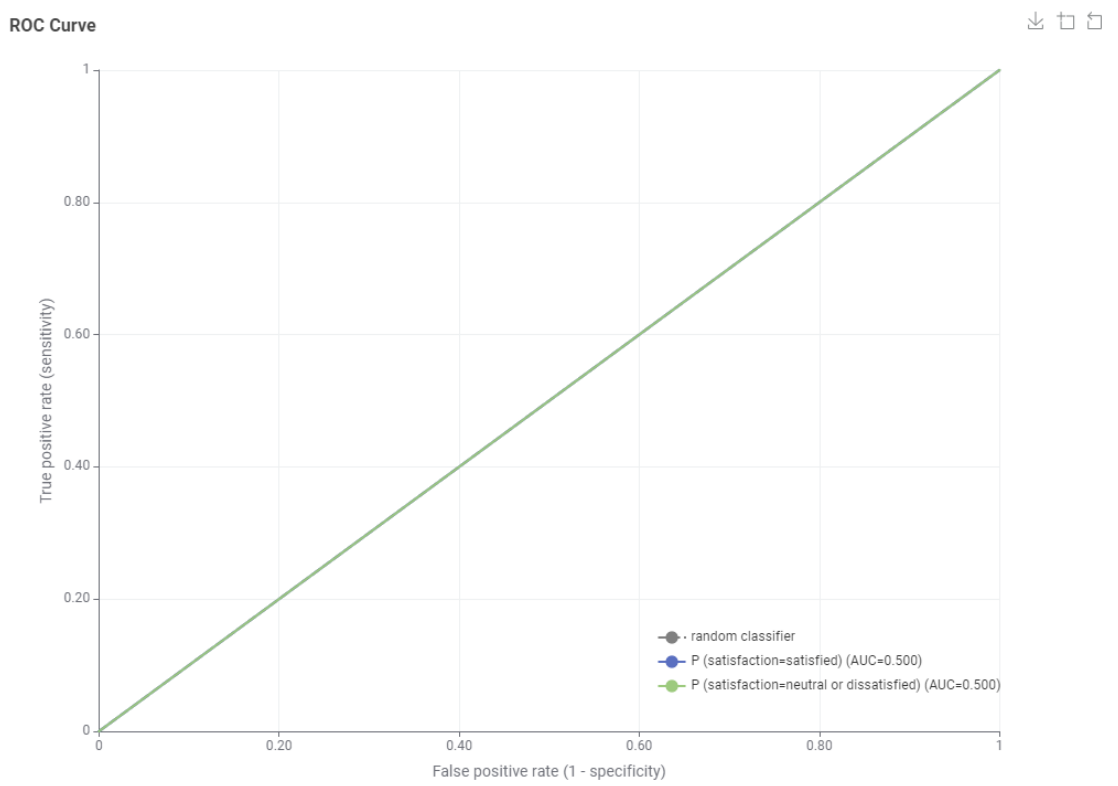
Table Statistics

#	RowID	TruePositives Number (Integer)	FalsePositives Number (Integer)	TrueNegatives Number (Integer)	FalseNegatives Number (Integer)	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's kappa Number (Float)
1	satisfie	10129	308	13823	677	0.937	0.97	0.937	0.978	0.954	?	?
2	neutral	13823	677	10129	308	0.978	0.953	0.978	0.937	0.966	?	?
3	Overall	?	?	?	?	?	?	?	?	?	0.961	0.919

It’s almost same as Random Forest.

Debugging model

In this assignment, I adjusted the model parameters multiple times (such as the sigma and C-value of SVM, the number of hidden layers and neurons in MLP, the number of neighbors in KNN, etc.), and repeatedly evaluated the model performance using cross validation (X-Partitioner and X-Aggregator nodes). But there may be many problems during the process, such as the one shown in the following figure:



#	RowID	TruePositives Number (Integer)	FalsePositives Number (Integer)	TrueNegatives Number (Integer)	FalseNegatives Number (Integer)	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's kappa Number (Float)
1	neutral	14131	10806	0	0	1	0.567	1	0	0.723	?	?
2	satisfa	0	0	14131	10806	0	?	0	1	?	?	?
3	Overall	?	?	?	?	?	?	?	?	?	0.567	0

During the model debugging process, the SVM model experienced a situation where it was unable to learn patterns. As shown in the figure, the ROC curve completely overlaps with the random classifier (AUC=0.5), and all samples are predicted as the same category with an accuracy of only 56.7% and a Kappa coefficient of 0. This problem is usually caused by improper parameter settings (such as sigma being too small or C value being too low), data not

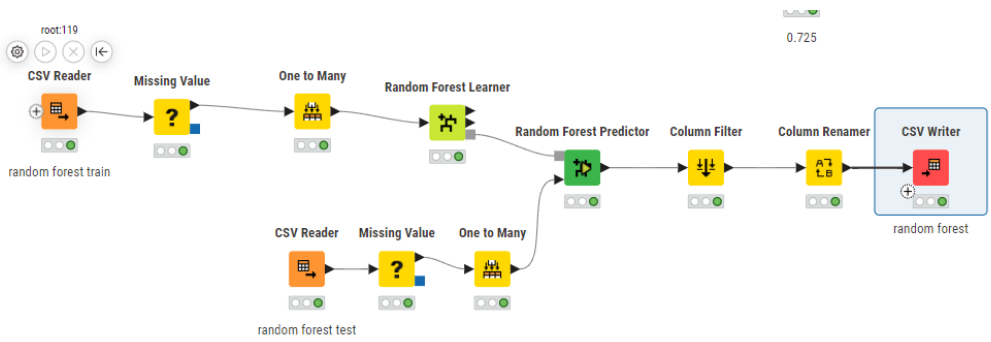
being standardized, or class imbalance. After adjusting the kernel function parameters and standardizing the input data, the model performance was significantly improved.

The best classifier

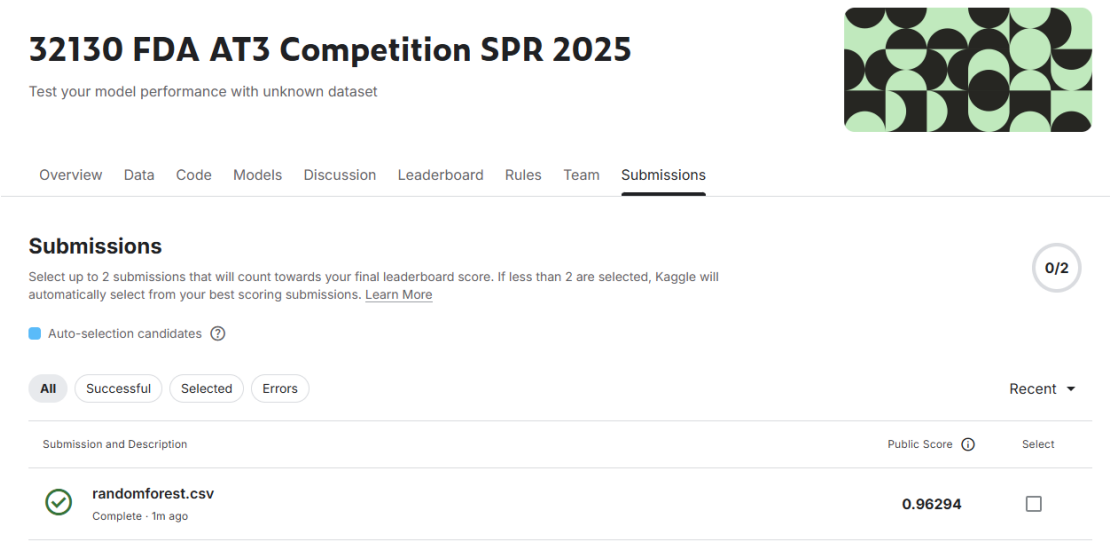
After comparing the AUC, accuracy, and stability of various models (such as Decision Tree, KNN, SVM, MLP, Random Forest) through cross validation, the model with the best performance and strong generalization ability was selected - in this experiment, Random Forest was the overall best performing classifier.

From the results, the accuracy of Random Forest reached 0.961 and the AUC was as high as 0.994, far higher than other models. This indicates that it has extremely high predictive ability in distinguishing between "satisfied" and "dissatisfied" customers. Random Forest effectively reduces the over fitting problem of a single model by integrating multiple decision trees (usually more than 100), using "Bagging" (self-help sampling) and random feature selection, and makes the model stable in cross validation. It is insensitive to outliers and noise data, and even if some features are irrelevant or of poor quality, the overall prediction accuracy will not significantly decrease. Unlike MLP or SVM, Random Forest does not require fine tuning of complex parameters such as learning rate and kernel function parameters, and can achieve high performance by default configuration.

KNIME workflow



Kaggle submission



Reflection

In this assignment, I used KNIME to construct and compare various classification models, including Random Forest, Decision Tree, kNN, SVM, and MLP. Among them, Random Forest left the deepest impression on me. It is based on the idea of ensemble learning, which reduces variance by performing bootstrap and bagging on multiple decision trees to obtain more stable and accurate prediction results. In practice, I have found that Random Forest not only effectively reduces overfitting problems, but also provides feature importance analysis to help understand which variables have the greatest impact on classification results. By comparing the confusion matrices and ROC-AUC of different models, I validated the advantages of Random Forest in terms of accuracy and generalization ability. This assignment has helped me better understand the ideas behind ensemble algorithms and enhanced my skills in using KNIME for data analysis and model evaluation.

Through this experiment, I not only mastered the entire process of using different algorithms for modeling and validation on the KNIME platform, but also deepened my understanding of model selection and parameter tuning strategies. During the debugging process, I found that different algorithms have significant differences in sensitivity to data preprocessing. For example, SVM and KNN rely more on standardization, while Decision Tree and Random Forest can maintain stable performance on partially standardized data. This made me realize that in practical projects, the difference in algorithm performance not only depends on the complexity

of the model itself, but also on the impact of data features and cleaning quality. In addition, I also appreciate the importance of cross validation in evaluating the generalization ability of models, as it can effectively avoid bias caused by data partitioning. Overall, this assignment has enhanced my understanding of the entire process of machine learning - from data preprocessing, feature encoding, model selection to result interpretation - giving me a more systematic analytical mindset and laying a solid foundation for more complex predictive modeling in the future.

References

IBM. (2021a, October 4). *What is the k-nearest neighbors (KNN) algorithm?* Ibm.com. https://www.ibm.com/think/topics/knn

IBM. (2021b, October 20). *What is random forest?* Ibm.com; IBM. https://www.ibm.com/think/topics/random-forest

IBM. (2021c, November 2). *What Is a Decision Tree?* IBM. https://www.ibm.com/think/topics/decision-trees

IBM. (2023, December 12). *Support Vector Machine*. IBM. https://www.ibm.com/think/topics/support-vector-machine

Jaiswal, S. (2024, February 7). *Multilayer Perceptrons in Machine Learning: A Comprehensive Guide*. Datacamp.com; DataCamp. https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning

scikit-learn. (2012). *1.11. Ensemble methods — scikit-learn 0.22.1 documentation*. Scikit-Learn.org. https://scikit-learn.org/stable/modules/ensemble.html